

Módulo 2 — Value Objects (Parte 1 de 3)

Curso: Arquitectura de Software Profesional con Clean Architecture, DDD y SOLID

Caso de uso guía: CU-CLI-001 — Alta de Cliente

Objetivo del módulo

Comprender los **Value Objects** como objetos de dominio definidos por sus valores, no por su identidad. Aprender a construirlos inmutables, con validaciones encapsuladas y comparación por valor. Al finalizar este módulo, el alumno habrá reemplazado los tipos primitivos de la entidad **Customer** por Value Objects especializados, consolidando un dominio robusto y expresivo.

En este curso implementamos los Value Objects usando **sealed record** de C# / .NET 9, aprovechando la igualdad estructural incorporada en los records.

Módulo 1: entidad con strings → Módulo 2: entidad con Value Objects (records) →
Módulo 3: excepciones propias

Temas del módulo

Tema 1	¿Qué es un Value Object?
Tema 2	Inmutabilidad
Tema 3	Igualdad por valor
Tema 4	Encapsulación de validaciones
Tema 5 — Parte 2	Método Create() en Value Objects
Tema 6 — Parte 2	Excepciones en Value Objects
Tema 7 — Parte 2	Record base ValueObject
Tema 8 — Parte 3	Implementación: FullName, LastName, Document
Tema 9 — Parte 3	Implementación: Email, Phone, Address, BirthDate
Tema 10 — Parte 3	Actividades prácticas



Ubicación en el proyecto

```
CommerX.sln
+---src
+---CommerX.Domain
+---Common
|   +---Entities
|   |   └─ BaseEntity.cs           ← ya existe (Módulo 1) – abstract class
|   +---ValueObjects
|   |   └─ ValueObject.cs         ← abstract record base (se crea en este módulo)
+---Customers
+---Entities
|   └─ Customer.cs                ← se actualiza al final del módulo
+---ValueObjects
└─ FullName.cs                    ← sealed record
└─ LastName.cs                    ← sealed record
└─ Document.cs                    ← sealed record
└─ Email.cs                       ← sealed record
└─ Phone.cs                       ← sealed record
└─ Address.cs                     ← sealed record
└─ BirthDate.cs                   ← sealed record
```

Tema 1 — ¿Qué es un Value Object?

1. ¿Qué es?

En Domain-Driven Design, un **Value Object** (objeto de valor) es un objeto del dominio que se define exclusivamente por sus **valores**, no por una identidad única. Dos Value Objects son iguales si y solo si todos sus valores son iguales, independientemente de si son instancias distintas en memoria.

Un Value Object no tiene **Id**. No tiene ciclo de vida propio. Es **immutable**: una vez creado, sus valores no cambian. Si el valor necesita cambiar, se crea un nuevo Value Object.

2. Analogía

Imaginemos un **billete de \$100**. No importa qué billete de \$100 tengamos en mano: todos tienen el mismo valor. Un billete de \$100 es idéntico a otro billete de \$100 en cualquier contexto económico. No existen dos "billetes de \$100 diferentes" desde el punto de vista del valor que representan.

El número de serie del billete sería su identidad (como una entidad), pero el *valor monetario* en sí mismo es un Value Object: **\$100**. Si queremos **\$200**, no modificamos el billete: tomamos dos billetes de **\$100** o creamos uno nuevo de **\$200**.

3. Características esenciales

- **Sin identidad**: no tiene **Id**. La igualdad se determina por los valores.
- **Immutable**: una vez creado no puede modificarse. Toda operación que "cambia" el valor crea uno nuevo.
- **Autovalidado**: valida sus propios datos en el momento de creación. No puede existir en estado inválido.
- **Encapsula semántica**: **Email** no es solo un string; es un string que cumple las reglas de un email válido.
- **Reutilizable**: el mismo **Email** puede usarse en **Customer**, **Supplier**, **Employee**, etc.

4. Entidad vs Value Object — comparación directa

	Entidad	Value Object
Tipo C#	<code>sealed class</code> : <code>BaseEntity</code>	<code>sealed record</code> : <code>ValueObject</code>
Identidad	Tiene <code>Id</code> único	No tiene <code>Id</code>
Igualdad	Por <code>Id</code> — dos objetos con mismo Id son iguales aunque tengan datos distintos	Por valores — dos records con mismos valores son iguales aunque sean instancias distintas
Mutabilidad	Mutable — sus atributos pueden cambiar	Inmutable — sus valores no cambian tras la creación
Ciclo de vida	Nace, se modifica, puede eliminarse	Nace y queda fijo. Si "cambia", se reemplaza por uno nuevo
Ejemplo	<code>Customer</code> — el mismo cliente aunque cambie su email	<code>Email</code> — "juan@mail.com" siempre es igual a "juan@mail.com"

5. ¿Por qué se usa?

Sin Value Objects, las validaciones de formato están dispersas en múltiples lugares: en el controlador, en el caso de uso, en la entidad. Con Value Objects, cada concepto del dominio encapsula su propia validación.

- La regla "un email válido debe contener @" vive *una sola vez* en `Email.Create()`.
- Si mañana la regla cambia, se modifica en un único lugar.
- Es imposible crear un `Email` inválido: el objeto garantiza su propia consistencia.
- El código comunica intención: `Email email` dice más que `string email`.

6. ¿Cómo encaja en la arquitectura?

Clean Architecture	Los Value Objects viven en la capa Domain , junto a las entidades. No conocen EF Core, la API ni ninguna tecnología externa.
DDD	Son bloques de construcción fundamentales del modelo de dominio. Expresan conceptos del negocio con precisión y semántica.
SOLID — SRP	Cada Value Object tiene una sola responsabilidad: representar y validar un único concepto del dominio.
SOLID — OCP	Para agregar validaciones nuevas se extiende el Value Object, no se modifica la entidad ni otros objetos.

Tema 2 — Inmutabilidad

1. ¿Qué es?

Un objeto es **immutable** cuando sus valores no pueden modificarse después de su creación. No existe ningún método ni propiedad que permita cambiar el estado interno del objeto una vez construido.

En C# / .NET 9, los **record** son la herramienta ideal para los Value Objects porque sus propiedades son de solo lectura por diseño. Combinado con un **constructor privado** y un método **Create()** estático, se obtiene inmutabilidad completa sin código extra.

2. Analogía

Una **coordenada GPS** es immutable: latitud **-34.6037**, longitud **-58.3816**. Esas coordenadas representan un punto específico. Si queremos representar otro punto, no modificamos las coordenadas existentes: creamos un nuevo objeto con los nuevos valores. El punto original sigue siendo el mismo, intacto, sin cambios.

3. Inmutabilidad en records — C# / .NET 9

```
1 // — Record con constructor privado — patrón que usaremos en este curso —
2 public sealed record Email : ValueObject
3 {
4     public string Value { get; } // solo get — immutable por diseño del record
5
6     private Email(string value) => Value = value;
7
8     public static Email Create(string? value)
9     {
10         // validaciones aquí (Tema 4)
11         return new Email(value!.Trim().ToLower());
12     }
13 }
```

```
1 // El constructor es privado y Value no tiene setter — ambas vías bloqueadas:
2 var email = Email.Create("juan@mail.com");
3 email.Value = "otro@mail.com"; // no compila — propiedad de solo lectura
4 new Email(); // no compila — constructor privado
```

4. ¿Por qué la inmutabilidad es importante en Value Objects?

- **Predecibilidad:** un `Email` creado con "juan@mail.com" siempre tendrá ese valor. Ningún código externo puede cambiarlo por error.
- **Seguridad de threads:** los objetos inmutables pueden compartirse entre hilos sin riesgo de condiciones de carrera.
- **Consistencia del dominio:** si el email del cliente cambia, se crea un nuevo `Email` validado — no se "parchea" el anterior.
- **Facilita los tests:** dado el mismo input, `Email.Create()` siempre produce el mismo output — es puro y determinista.

5. Inmutabilidad y reemplazo

Cuando el cliente cambia su email, la entidad `Customer` no modifica el Value Object existente: lo **reemplaza** con uno nuevo. Este es el patrón correcto en DDD:

```
1 // Intentar modificar el Value Object directamente - no compila:
2 customer.Email.Value = "nuevo@mail.com"; // propiedad de solo lectura en record
3
4 // Correcto: la entidad expone un método que reemplaza el VO por uno nuevo validado
5 public void UpdateEmail(string newEmail)
6 {
7     // Email.Create() valida el nuevo valor antes de asignarlo
8     Email = Email.Create(newEmail); // el record anterior es descartado, nace uno nuevo
9 }
```

Tema 3 — Igualdad por valor

1. ¿Qué es?

Los Value Objects implementan **igualdad estructural**: dos instancias son iguales si todos sus valores internos son iguales, sin importar si son el mismo objeto en memoria o instancias distintas.

En C# / .NET 9, los **record** implementan igualdad estructural de forma nativa: el compilador genera automáticamente **Equals()**, **GetHashCode()** y los operadores **==** / **!=** basándose en las propiedades declaradas. No es necesario escribir ningún código adicional para conseguirlo.

2. Analogía

Dos personas tienen escrito "juan@mail.com" en papeles distintos. Son dos papeles físicos diferentes (dos referencias distintas en memoria), pero el *contenido* escrito en ambos es idéntico. Desde el punto de vista del dominio, representan el mismo email. La igualdad por valor dice que son **equivalentes**.

3. Igualdad por valor automática en records

```
1 // Dos instancias creadas con el mismo valor son iguales - automáticamente
2 var email1 = Email.Create("juan@mail.com");
3 var email2 = Email.Create("juan@mail.com");
4
5 email1 == email2;           // true - operator == generado por el compilador
6 email1.Equals(email2);     // true - Equals() generado por el compilador
7 email1.GetHashCode() == email2.GetHashCode(); // true
8
9 var email3 = Email.Create("otro@mail.com");
10 email1 == email3;          // false - valores distintos
```

4. Lo que el compilador genera automáticamente en un record

Miembro generado	Comportamiento
<code>Equals(object?)</code>	Compara cada propiedad por valor
<code>GetHashCode()</code>	Hash basado en el valor de cada propiedad
<code>operator ==</code> y <code>!=</code>	Delegan a <code>Equals()</code> — comparan por valor
<code>ToString()</code>	Muestra el tipo y los valores de las propiedades (útil en logs y depuración)

Todo este comportamiento es generado por el compilador de C# 9+ sin ninguna instrucción adicional. Los records están diseñados para representar exactamente este tipo de objetos: datos inmutables definidos por sus valores.

5. ¿Por qué es importante en el dominio?

- Permite comparar emails, documentos o nombres directamente con `==` sin sorpresas.
- Al buscar duplicados (ej: email único), la comparación es semánticamente correcta.
- Funciona correctamente en colecciones como `HashSet<Email>` y `Dictionary<Email, ...>`.
- No hay riesgo de comparaciones incorrectas: el compilador lo garantiza sobre todas las propiedades.

Tema 4 — Encapsulación de validaciones

1. ¿Qué es?

Encapsular validaciones significa que cada Value Object es responsable de verificar que su propio valor sea válido. Ningún código externo necesita saber cómo se valida un email: eso es responsabilidad exclusiva de `Email`.

Esta es la diferencia fundamental entre usar `string email` y `Email email` en el dominio. El tipo `string` no dice nada sobre su contenido. El tipo `Email` garantiza que su contenido es un email válido.

2. Analogía

Un **cajero automático** verifica que el billete que ingresás sea auténtico antes de aceptarlo. No necesitás saber cómo funciona el detector de billetes: el cajero lo maneja internamente. Si el billete no pasa la verificación, simplemente lo rechaza. El Value Object funciona igual: verifica internamente y rechaza si el dato no es válido.

3. Validación dispersa vs encapsulada

```
1 // Sin Value Objects – la misma regla duplicada en múltiples capas:
2 // En el controlador:
3 if (!email.Contains("@")) return BadRequest("Email inválido");
4 // En el caso de uso:
5 if (!email.Contains("@")) throw new Exception("Email inválido");
6 // En la entidad:
7 if (string.IsNullOrEmpty(email) || !email.Contains("@"))
8     throw new ArgumentException("Email inválido");
9 // tres lugares distintos, tres versiones distintas, tres posibles bugs
```

```
1 // Con Value Objects – la regla vive una sola vez en Email.Create():
2 public sealed record Email : ValueObject
3 {
4     public string Value { get; }
5     private Email(string value) => Value = value;
6
7     public static Email Create(string? value)
8     {
9         if (string.IsNullOrEmpty(value))
10             throw new ArgumentException("El email es obligatorio.", nameof(value));
11         if (!value.Contains("@"))
12             throw new ArgumentException("El email debe contener @.", nameof(value));
13         return new Email(value.Trim().ToLower());
14     }
15 }
16 }
```

```

17 // En Customer – delega toda la validación al record:
18 Email = Email.Create(email);
19 // En el controlador o caso de uso – tampoco repiten nada
20 // Si llega un email inválido, la excepción viaja desde Email.Create() hacia arriba

```

4. SOLID — SRP aplicado a Value Objects

Cada Value Object tiene una **única razón para cambiar**: cuando cambia la regla de validación de ese concepto.

Email	Cambia solo si cambia la regla de validación del email.
Document	Cambia solo si cambia el formato válido del documento.
BirthDate	Cambia solo si cambia la regla de mayoría de edad.
Customer	Ya no necesita conocer el formato de un email. Eso es responsabilidad de Email.

Resumen del Tema 4

- Con `sealed record`, la entidad `Customer` ya no valida formatos: los delega.
- Cada VO es una cápsula que garantiza que su valor es siempre válido.
- La regla de negocio vive en un único lugar: el `Create()` del record.
- Si la regla cambia (ej: el email debe tener al menos 5 caracteres antes del @), se modifica solo en `Email.Create()`.

Continúa en **Módulo 2B** — Método `Create()` en Value Objects, Excepciones y record base `ValueObject`.